
title: Using the AWS CLI in Telekom environment

description: In this cookbook we describe how the AWS CLI can be used in the Telekom environment.

```
import useBaseUrl from '@docusaurus/useBaseUrl';
```

The AWS CLI is the [command line interface to AWS](#). It can be used to access and manage your resources in AWS in a programmatic way.

Setting up and Using the AWS CLI on a Telekom client including federated access

The steps for installing AWS CLI on a standard Telekom client without local administrative permissions are described here:

<https://yam-united.telekom.com/workspaces/aws-community/apps/blog/blog/view/5ea4d661-7559-4ac6-98f4-ed2b4453126e>

These instructions will work for Germany. WinPython is installed via Software Center and proxy can be configured again via SIA (special internet access). If you have more permissions on your machine you can install WinPython right away.

Using AWS CLI as an External user

Whereas the yam blog contains instructions for CLI usage with ADFS the following instructions show you how you can use AWSCLI for a programmatic user in dtit-iam, which is protected by your MFA. Those are the instructions for users who are not represented in Telekom AD, e.g. external developers. For all internal employees the instructions in the blog entry apply as they do not have a user in dtit-iam account.

Install Python and AWSCLI

Linux

- Download and install python (2.7 or above / 3.6 or above). The latest versions of python come also with the PIP package manager installed.

Windows

- Install WinPython

Install AWSCLI with command: `$ pip install awscli`

Generate Access Key

- Click Services, choose IAM
- Click Users in the left navigation pane
- Search for your user by your username (e-mail address)
- Find your user

Create access key

After creating your access key do not forget to download it. You will need it when configuring the awscli.

Configure CLI and set up MFA

1. Check if aws is installed `$ aws --version`
2. Install the aws-mfa package with pip `$ pip install aws-mfa`
3. Configure awscli using the keys you generated in step 2

Add the following to `~/.aws/credentials`: `[dtit-iam] aws_access_key_id = <access key> aws_secret_access_key = <secret access key> aws_mfa_device = arn:aws:iam::315005456645:mfa/<email address>`

Replace `<access key>`, `<secret access key>` and `<email address>` (user name) with the correct values.

Add the following to `~/.aws/config`: `[profile dtit-nonprod] region=eu-central-1 source_profile = dtit-iam role_arn = arn:aws:iam::<account>:role/DTIT_Project_Admin`

Replace `<account>` with the correct value for the account in which you wish to assume the admin role.

The profile name can be anything you want. You can use multiple blocks to assume roles in different accounts.

1. Run `aws-mfa` to generate an STS token for the IAM account `aws-mfa --profile dtit-iam`
2. Set the `AWS_DEFAULT_PROFILE` environment variable to match the profile you wish to assume

Windows PowerShell: `$Env:AWS_DEFAULT_PROFILE = "dtit-nonprod"`

Linux / macros: `export AWS_DEFAULT_PROFILE=dtit-nonprod`

1. Fill the parameters, then list out the s3 buckets to test the access `aws sts get-caller-identity` `aws s3 ls`